

Intermediate (build the gate) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real release gate refuses to be an easy-to-skip formality). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A release gate is one function: try to make a version "live," but only actually do it if that version's test score is good enough. If it isn't, print why, and don't touch anything.

Things to use:

- A function `deploy(version_name, threshold)`.
- Your `run_eval_suite(prompt_text)` stub from the `version_log` exercise, to get a score.
- An `if score >= threshold:` check.
- Writing the winning version's name into a plain text file, `live_version.txt`, only inside that `if`.

Intermediate Version

The whole exercise is really just one `if / else` with real consequences on each side. `deploy()` looks up the version (from the log you built in `version_log`), runs the test suite against its prompt text, and compares the score to `threshold`. If it passes, it writes the version name to `live_version.txt` — that file *is* your "what's currently live" pointer, the thing every other part of the system reads to know which prompt to actually use. If it fails, `deploy()` must not touch `live_version.txt` at all — not write an empty string, not write anything — since any write at all means it changed state on a rejected release.

The exact pieces:

- Your `load_versions(path)` and `run_eval_suite(prompt_text)` from `version_log`, reused here — look the target version up by name out of the loaded list.
- `open("live_version.txt", "w") as f: f.write(version_name)` — only inside the passing branch.
- A return value or printed message on failure that names the score and the threshold, e.g. `f"Score {score} below threshold {threshold} – not deploying."` — a gate that fails silently is nearly as bad as one that doesn't exist.
- Reading the current live version back: `open("live_version.txt").read().strip()`.

Advanced Version

The real design question isn't "how do I write an if-statement," it's **what does `deploy()` return, and what can the caller (or a human reading the terminal) actually learn from it?** A function that just prints something and returns `None` is hard to test and hard to build other tools on top of — and this document's whole Build Task is built on top of exactly this function. Return a small, structured result instead — a `DeployResult` with `passed: bool`, `score: float`, and `failing_cases: list` — so both a human and later code (the Build Task's `deploy_gate.py`) can act on it without re-parsing printed text.

The other real design question: **what happens if `deploy()` is called with a version name that was never saved in the version log?** Looking it up and getting `None` back, then trying to read `.prompt_text` off of `None`, is exactly the "confusing failure somewhere later" problem Doc01's config loader was built to avoid. Check explicitly, and raise a clear, named error (`VersionNotFoundError`) instead of letting a missing lookup blow up as an unrelated `AttributeError`.

The extra pieces needed:

- A small result type — either a `@dataclass DeployResult` or, simplest, a dict with a fixed shape (`{"passed": bool, "score": float, "failing_cases": list}`).
- `class VersionNotFoundError(Exception): pass`, raised the moment the looked-up version comes back missing — before anything else runs.
- Keep `live_version.txt` untouched on any failure path, including this new one — a version that doesn't exist obviously can't become live.

Sketch `DeployResult` and the not-found check yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the one `if / else` and the two tools (test suite, text file) needed for the tidy case where the version always exists. Intermediate shows the real Python for reading the log, writing the pointer only on success, and printing a clear reason on failure. Advanced adds what matters once other code (the Build Task) needs to build on top of this function instead of a human just watching it print — a structured return value instead of only printed text, and a named error for a version name that was never actually saved.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
function deploy(version_name, threshold):
    versions = load all saved versions
    find the one matching version_name
    result = run_eval_suite(that version's prompt_text)
    if result's score >= threshold:
        write version_name into live_version.txt
        print "deployed"
    else:
        print "score too low, not deploying" with the actual score
```

Here's almost the whole thing — just try running it and reading it line by line:

```
def deploy(version_name, threshold):
    versions = load_versions("versions.json")
    target = None
    for v in versions:
        if v["version_name"] == version_name:
            target = v

    result = run_eval_suite(target["prompt_text"])
    if result["score"] >= threshold:
        with open("live_version.txt", "w") as f:
            f.write(version_name)
        print(version_name, "deployed, score", result["score"])
    else:
        print(version_name, "NOT deployed, score", result["score"], "below thresh
```

Expected output, calling `deploy("v2", 0.9)` then `deploy("v1", 0.9)` against the `versions.json` from `version_log`:

```
v2 deployed, score 1.0
v1 NOT deployed, score 0.6 below threshold 0.9
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
function deploy(version_name: str, threshold: float) -> None:
    versions = load_versions("versions.json")
    target = find the dict in versions where version_name matches
    if target is None:
        print "no such version", return

    result = run_eval_suite(target["prompt_text"])
    if result["score"] >= threshold:
        write version_name to live_version.txt
        print a success message with the score
    else:
        print a failure message with the score and threshold, do NOT write the fi
```

```
def find_version(versions: list[dict], name: str) -> dict | None:
    for v in versions:
        if v["version_name"] == name:
            return v
    return None

def deploy(version_name: str, threshold: float) -> None:
    versions = load_versions("versions.json")
    target = find_version(versions, version_name)
    if target is None:
        print(f"No such version: {version_name}")
        return

    result = run_eval_suite(target["prompt_text"])
    if result["score"] >= threshold:
        with open("live_version.txt", "w") as f:
            f.write(version_name)
        print(f"{version_name} deployed – score {result['score']} >= {threshold}")
    else:
        print(f"{version_name} NOT deployed – score {result['score']} < {threshold}")
```

Now add a `get_live_version()` function that reads `live_version.txt` back, and compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
class VersionNotFoundError is an Exception

function deploy(version_name, threshold) -> DeployResult:
  versions = load_versions("versions.json")
  target = find_version(versions, version_name)
  if target is None:
    raise VersionNotFoundError naming version_name

  result = run_eval_suite(target["prompt_text"])
  passed = result["score"] >= threshold

  if passed:
    write version_name to live_version.txt

  return DeployResult(passed, result["score"], result.get("failed", []))
```

Here's almost the whole thing — fill in `DeployResult` yourself:

```
class VersionNotFoundError(Exception):
    pass

# your turn: define DeployResult, either as a @dataclass with fields
# passed: bool, score: float, failing_cases: list – or a plain dict
# with those same 3 keys

def deploy(version_name: str, threshold: float) -> "DeployResult":
    versions = load_versions("versions.json")
    target = find_version(versions, version_name)
    if target is None:
        raise VersionNotFoundError(f"No saved version named: {version_name}")

    result = run_eval_suite(target["prompt_text"])
    passed = result["score"] >= threshold

    if passed:
        with open("live_version.txt", "w") as f:
            f.write(version_name)
```

```
# your turn: build and return a DeployResult from passed, result["score"],  
# and result.get("failed", [])
```

Fill in `DeployResult` and the return statement yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same underlying idea (look up, test, compare, maybe write) at 3 completeness levels — Basic prints the outcome and returns nothing, Intermediate adds a real not-found guard with an early `return` and full type hints, and Advanced replaces "look up returns `None`, silently skipped" with a raised `VersionNotFoundError`, and replaces "outcome only exists as printed text" with a real `DeployResult` object other code (the Build Task) can act on.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)